

● RAPPORT DE PROJET FINAL · E4FG SI

Adopte un Compagnon

Site web et base de données — plateforme d'adoption animale.

MODULE

E4FG — SI — ESIEE — 2026

BINÔME

Rihane Mahroug & Inès Ben Fakir

SUJET

Plateforme d'adoption animale

SITE EN LIGNE

ines-rihane-adoption-animal.site.je

Présentation

Notre site s'appelle *Adopte un Compagnon*. C'est une plateforme d'adoption animale qui met en relation des refuges et des personnes qui veulent adopter. Les refuges (et parfois des particuliers) publient des annonces d'animaux à adopter, et les visiteurs peuvent les consulter, chercher un animal selon plusieurs critères, puis envoyer une demande d'adoption.

Sur le site, il y a quatre types d'utilisateurs connectés. Un adoptant cherche des animaux et envoie des demandes. Un refuge publie et gère ses annonces, et il traite les demandes qu'il reçoit. Un particulier fait pareil mais pour ses propres animaux, sans passer par un refuge. Un administrateur valide les nouveaux refuges et garde un œil sur l'ensemble.

On a choisi ce sujet parce qu'il nous laissait manipuler des données assez variées et plusieurs liens entre les tables, tout en couvrant ce qui était demandé dans le projet.

Mini-documentation utilisateur

Objectif du site

Le site sert à consulter, publier et gérer des annonces d'adoption d'animaux. Un visiteur peut chercher un animal par espèce, par sexe ou par ville, regarder où sont les refuges sur une carte, et envoyer une demande d'adoption. Un refuge s'occupe de ses annonces et de ses demandes, et un administrateur supervise le tout.

Les rôles

Il y a cinq profils sur le site. Le visiteur n'a pas de compte : c'est juste quelqu'un qui arrive sur le site et le regarde. Les autres ont besoin d'un compte, et le menu change tout seul selon le rôle.

Rôle	Compte	Ce qu'il peut faire
Visiteur	aucun (accès libre)	Voir l'accueil, chercher des animaux, ouvrir les fiches et la carte des refuges. Il ne peut pas faire de demande tant qu'il n'est pas connecté.
Adoptant	email + mot de passe	Comme le visiteur, et en plus il envoie des demandes d'adoption et suit ses demandes dans « Mes demandes ».
Particulier	email + mot de passe	Proposer ses propres animaux à l'adoption (sans passer par un refuge), les modifier ou les supprimer, et accepter ou refuser les demandes reçues sur ses animaux.
Refuge	email + mot de passe	Publier, modifier et supprimer ses annonces (avec photo), et accepter ou refuser les demandes reçues sur ses animaux. Un refuge doit d'abord être validé par un administrateur.
Administrateur	email + mot de passe	Valider ou refuser les nouveaux refuges, voir toutes les demandes du site et consulter des statistiques.

Accès au site

Site : ines-rihane-adoption-animal.site.je

Six comptes de test sont disponibles, un par rôle. Le mot de passe est le même pour tous : `password`. Les comptes `ines` et `rihane` sont les nôtres (les deux membres du binôme).

Rôle	Identifiant (email)	Mot de passe
Administrateur	<code>admin@adoption.fr</code>	<code>password</code>
Administrateur	<code>ines@adoption.fr</code>	<code>password</code>
Refuge	<code>refuge@adoption.fr</code>	<code>password</code>
Adoptant	<code>adoptant@adoption.fr</code>	<code>password</code>
Adoptant	<code>rihane@adoption.fr</code>	<code>password</code>
Particulier	<code>sophie@adoption.fr</code>	<code>password</code>

Pour utiliser le site, on commence sur la page d'accueil. Elle montre le projet, les six catégories d'animaux et une carte des refuges. On peut regarder les animaux sans être connecté, en passant par le menu « Adopter » ou en cliquant sur une catégorie. Pour faire une demande d'adoption, il faut être connecté. Un refuge retrouve ses annonces et ses demandes dans le menu une fois connecté, et l'administrateur a une page « Administration ».

Plan

- | | |
|--|---|
| 01 Introduction générale | 08 Gestion des demandes |
| 02 Architecture technique du site | 09 Administration |
| 03 Inscription et validation des refuges | 10 Procédure de lancement |
| 04 Connexion | 11 Choix techniques et sécurité |
| 05 Recherche et consultation des animaux | 12 Organisation et répartition du travail |
| 06 Publier une annonce | 13 Apports de l'intelligence artificielle |
| 07 Demande d'adoption | 14 Conclusion |

1 Introduction générale

Dans le cadre du module E4FG à l'ESIEE, on a réalisé un projet qui mélange le développement web et la gestion d'une base de données. Pour mettre en pratique ce qu'on a vu en HTML, CSS, PHP et MySQL, on a fait une plateforme d'adoption animale.

L'idée de départ est simple : comment relier facilement les refuges qui cherchent à faire adopter des animaux et les gens qui veulent en adopter ? Pour répondre à ça, on a fait un site où les refuges publient leurs annonces et où les adoptants envoient leurs demandes, avec un administrateur qui valide les refuges.

Ce projet nous a fait travailler plusieurs choses concrètes : l'organisation d'un site dynamique avec une partie affichage et une partie traitement, la sauvegarde des données dans une base MySQL, la sécurité de la connexion, et l'aspect de l'interface pour qu'elle reste claire et utilisable sur mobile comme sur ordinateur.

Ce rapport présente les étapes de conception et de réalisation du projet.

2 Architecture technique du site

2.1 Structure des fichiers

L'application est rangée dans un dossier `htdocs/` pour qu'elle corresponde directement à la racine d'un serveur web. Chaque page est un fichier, et ce qui est commun est mis à part.

ARBORESCENCE

```
htdocs/
  connexion.php          # connexion PDO (un seul endroit, detection auto
local/en-ligne)
  index.php              # accueil : hero, categories, carte des refuges, derniers
animaux
  recherche.php          # recherche filtree des animaux
  animal.php             # fiche detaillee d'un animal
  login.php              # connexion
  register.php           # inscription (adoptant ou refuge)
  logout.php             # deconnexion
  mes-annonces.php       # tableau de bord refuge
  annonce-form.php       # ajout / modification d'une annonce (+ photo)
  supprimer-annonce.php  # suppression d'une annonce
  demande-adoption.php   # formulaire de demande d'adoption
  mes-demandes.php       # demandes envoyees + recues (accepter/refuser)
  admin.php              # back-office : validation refuges, stats, demandes
  includes/              # header.php, footer.php, auth.php, functions.php
  css/style.css          # feuille de style
  img/                  # photos des animaux, image d'accueil, dossier uploads/
```

Le découpage est simple : un fichier par page, et les morceaux communs comme l'en-tête, le pied de page, les fonctions utilitaires et la connexion à la base sont mis dans le dossier `includes/` et dans `connexion.php`. Les identifiants de la base ne sont écrits qu'à un seul endroit, ce qui rend les tests plus faciles.

2.2 Architecture client-serveur

Le site marche sur une architecture web classique de type client-serveur. Ça sépare bien le rôle du client (l'interface) et celui du serveur (le traitement et la base).

Le client, c'est le navigateur de l'utilisateur. Il affiche les pages faites par le serveur et récupère les actions à travers les formulaires. Il ne touche jamais directement à la base : tout passe par des requêtes envoyées au serveur.

Le serveur, c'est Apache avec PHP (sous XAMPP en local, et chez l'hébergeur en ligne). Il reçoit les requêtes et exécute les scripts PHP. Ces scripts traitent les formulaires, se connectent à la base MySQL avec PDO, créent des sessions pour la connexion, et fabriquent le HTML à partir des données de la base.

Pour la connexion à la base, on a regroupé la configuration dans `connexion.php`. Le fichier reconnaît tout seul l'environnement, comme ça le même code marche en local et en ligne sans qu'on touche à rien.

PHP — CONNEXION.PHP

```
$host = $_SERVER['HTTP_HOST'] ?? '';
$enLigne = (strpos($host, 'infinityfree') !== false)
    || (strpos($host, '.site.je') !== false);

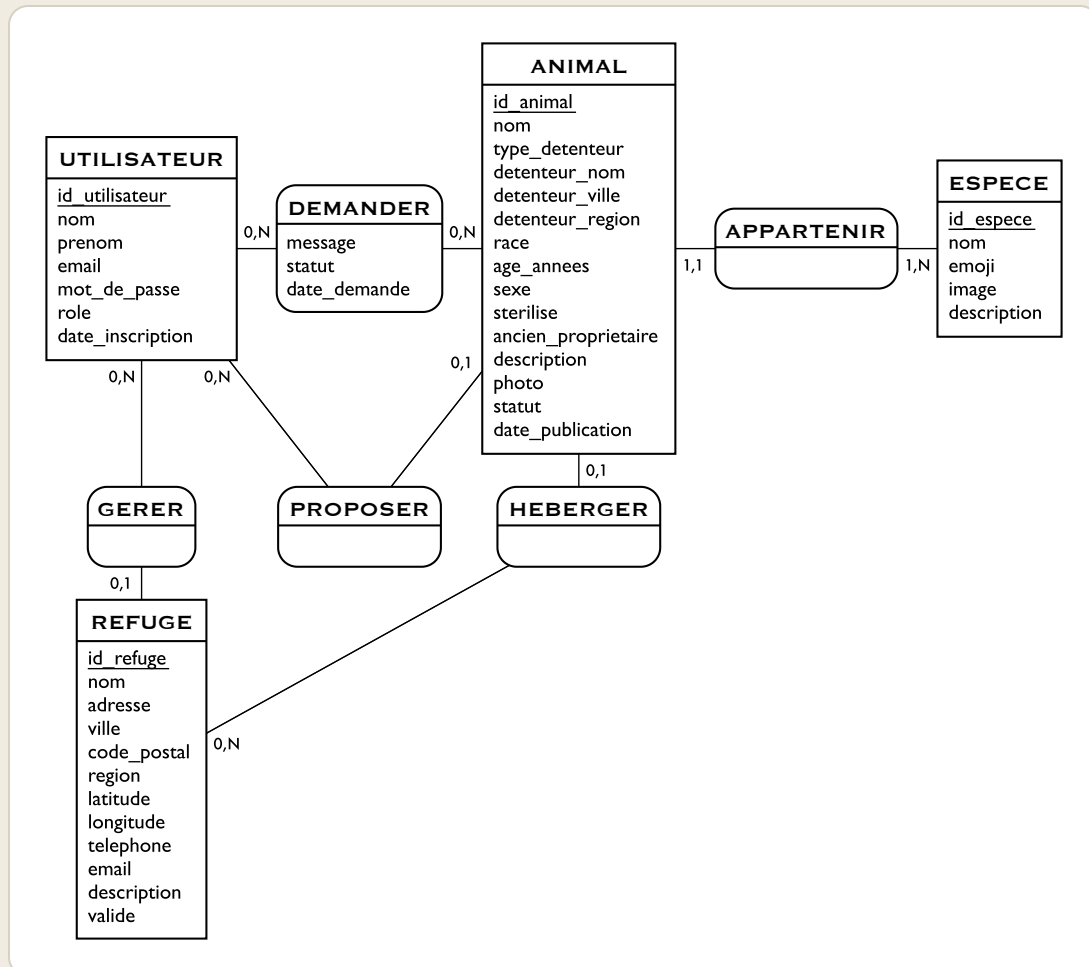
if (!$enLigne) {
    // Environnement local (XAMPP)
    $DB_HOST = 'localhost'; $DB_NAME = 'adoption';
    $DB_USER = 'root';      $DB_PASS = '';
} else {
    // Environnement en ligne
    $DB_HOST = 'sql311.infinityfree.com';
    $DB_NAME = 'if0_42217380_adoption';
    // ...
}

$pdo = new PDO("mysql:host=$DB_HOST;dbname=$DB_NAME;charset=utf8mb4",
    $DB_USER, $DB_PASS,
    [ PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
      PDO::ATTR_EMULATE_PREPARES => false ]);
```

Ce code regarde le nom de domaine de la requête. Si ce n'est pas celui de l'hébergement, il prend les paramètres de XAMPP (utilisateur `root`, mot de passe vide), sinon ceux de l'hébergeur. L'option `EMULATE_PREPARES => false` demande à PDO d'utiliser de vraies requêtes préparées, ce qui aide contre les injections SQL.

2.3 Base de données

La première étape a été de concevoir la base. On est partis d'un schéma entité-association, puis on a écrit le schéma relationnel qui va avec.



Modèle conceptuel des données (schéma entité-association).

Le schéma entité-association ci-dessus représente les données du site. Il y a quatre entités principales : les utilisateurs, les espèces, les refuges et les animaux. Un utilisateur peut gérer un refuge (un refuge étant géré par zéro ou un utilisateur). Un animal appartient à une seule espèce. Un animal peut être hébergé par un refuge, mais pas toujours : la cardinalité 0,1 veut dire qu'un animal peut aussi être proposé par un particulier, donc sans refuge. C'est l'association *proposer* entre l'utilisateur et l'animal : un utilisateur particulier peut proposer plusieurs animaux, et un animal est proposé par zéro ou un particulier. Enfin, un utilisateur peut demander l'adoption de plusieurs animaux. Cette dernière association porte des informations (le message, le statut et la date), donc elle devient une table à part quand on passe au relationnel.

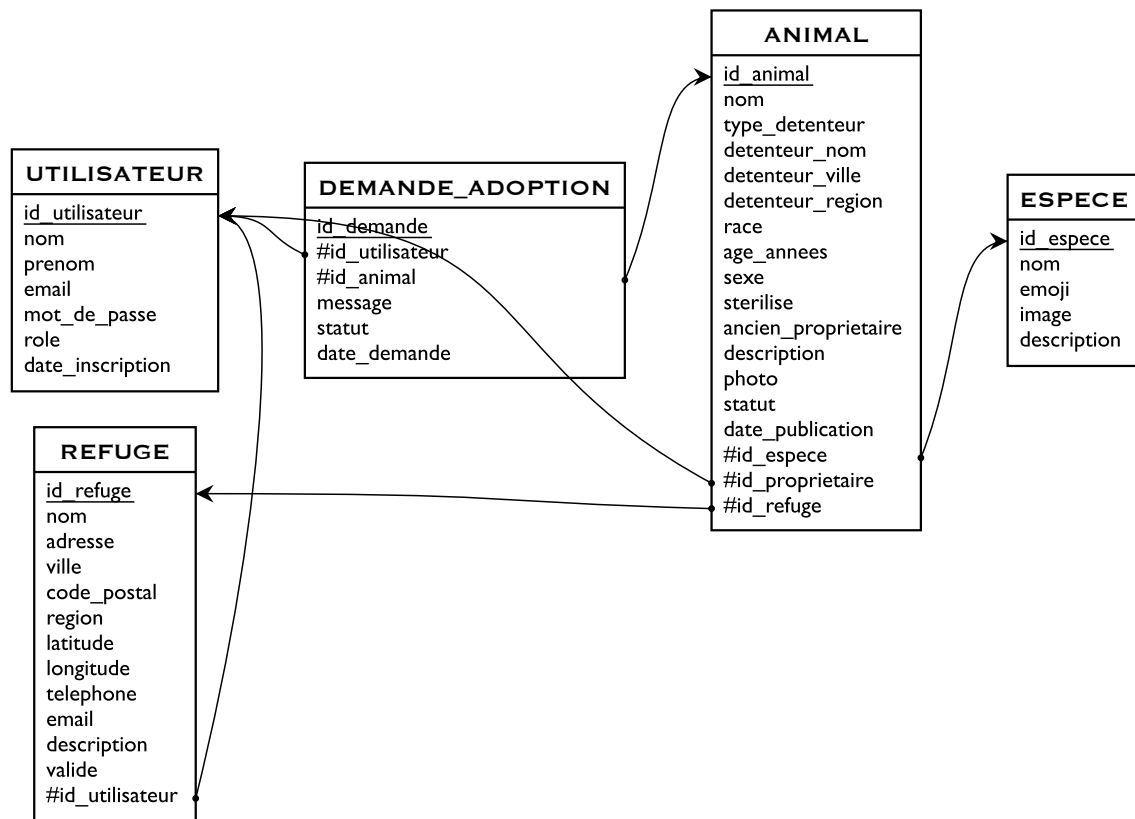


Schéma relationnel — cinq tables, clés primaires et clés étrangères.

En passant au relationnel, on obtient cinq tables. Les clés primaires identifient chaque ligne, et les clés étrangères (le dièse) font le lien entre les tables.

SCHÉMA RELATIONNEL

```

utilisateur(id_utilisateur, nom, prenom, email, mot_de_passe, role,
date_inscription)
espece(id_espece, nom, emoji, image, description)
refuge(id_refuge, nom, adresse, ville, code_postal, region, latitude, longitude,
telephone, email, description, valide, #id_utilisateur)
animal(id_animal, nom, #id_espece, type_detenteur, #id_refuge, #id_proprietaire,
detenteur_nom, detenteur_ville, detenteur_region, race, age_annees, sexe,
sterilise, ancien_proprietaire, description, photo, statut, date_publication)
demande_adoption(id_demande, #id_animal, #id_utilisateur, message, statut,
date_demande)
  
```


On a fait quelques choix au moment de la conception. Les colonnes comme `role`, `sexe`, `statut` ou `type_detenteur` sont en type ENUM : ça limite les valeurs possibles directement dans la base et ça évite les erreurs. Le `role` peut valoir `admin`, `refuge`, `visiteur` (un adoptant) ou `particulier`. Un animal est détenu soit par un refuge, soit par un particulier. Dans le premier cas la colonne `id_refuge` est remplie ; dans le second elle est laissée vide (NULL) et c'est la colonne `id_proprietaire` qui pointe vers le compte du particulier (avec `detenteur_nom`, `detenteur_ville` et `detenteur_region` pour l'affichage). C'est pour ça que `id_refuge` et `id_proprietaire` peuvent être nuls selon le cas. La colonne `valide` du refuge sert à la validation par l'administrateur : un refuge qui vient de s'inscrire reste à 0 tant qu'il n'est pas approuvé. Enfin, les coordonnées `latitude` et `longitude` du refuge servent à l'afficher sur la carte de l'accueil.

Voici un exemple de création de table, repris du dump de la base :

SQL

```
CREATE TABLE demande_adoption (
  id_demande      INT AUTO_INCREMENT PRIMARY KEY,
  id_animal       INT NOT NULL,
  id_utilisateur  INT NOT NULL,
  message         TEXT DEFAULT NULL,
  statut          ENUM('en_attente','acceptee','refusee') NOT NULL DEFAULT
'en_attente',
  date_demande    DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_dem_animal FOREIGN KEY (id_animal)
    REFERENCES animal(id_animal) ON DELETE CASCADE,
  CONSTRAINT fk_dem_user FOREIGN KEY (id_utilisateur)
    REFERENCES utilisateur(id_utilisateur) ON DELETE CASCADE
) ENGINE=InnoDB;
```

Les contraintes `ON DELETE CASCADE` font qu'une demande est supprimée automatiquement si l'animal ou l'utilisateur lié est supprimé. Comme ça on ne garde pas de lignes toutes seules dans la base.

2.4 Flux de navigation

Le parcours est assez simple. L'utilisateur arrive sur la page d'accueil, qui montre le projet, les catégories d'animaux et la carte des refuges. À partir de là, il peut regarder les animaux sans être connecté, ou bien s'inscrire et se connecter. Une fois connecté, une session est ouverte et le menu

s'adapte au rôle : un refuge voit « Mes annonces » et « Mes demandes », un adoptant voit « Mes demandes », et un administrateur voit « Administration ». On peut fermer la session à tout moment avec la déconnexion.

3 Inscription et validation des refuges

3.1 Flux utilisateur

L'inscription se fait sur la page `register.php`. L'utilisateur entre son nom, son prénom, son email et un mot de passe, puis il choisit son type de compte : adoptant, particulier ou refuge. S'il choisit refuge, deux champs en plus apparaissent pour le nom et la ville du refuge. Quand le formulaire est envoyé, les données sont vérifiées côté serveur, puis enregistrées. Pour un refuge, on crée en plus un refuge en attente de validation : tant qu'un administrateur ne l'a pas validé, ce compte ne peut pas publier d'annonce. Un particulier, lui, peut publier ses animaux directement après l'inscription, sans validation.

3.2 Requêtes serveur

Au début du traitement, on vérifie que les champs sont bons.

PHP — VALIDATION

```
if ($nom === '' || $prenom === '')           $erreurs[] = 'Le nom et le prenom sont
obligatoires.';
if (!filter_var($email, FILTER_VALIDATE_EMAIL)) $erreurs[] = 'Adresse email
invalide.';
if (strlen($mdp) < 6)                         $erreurs[] = 'Le mot de passe doit
faire au moins 6 caracteres.';
if ($mdp !== $mdp2)                          $erreurs[] = 'Les deux mots de passe ne
correspondent pas.';
```

Ces lignes contrôlent que le nom et le prénom sont remplis, que l'email a une forme correcte, que le mot de passe est assez long et qu'il est saisi deux fois pareil. Si une condition n'est pas remplie, on ajoute un message d'erreur et l'inscription ne se fait pas.

Si tout est bon, l'utilisateur est ajouté dans la base. Le mot de passe n'est jamais enregistré en clair : on le hache avant.

PHP — INSERTION UTILISATEUR

```
$ins = $pdo->prepare("INSERT INTO utilisateur (nom, prenom, email, mot_de_passe,
role)
                        VALUES (:nom, :prenom, :email, :mdp, :role)");

$ins->execute([
    'nom' => $nom, 'prenom' => $prenom, 'email' => $email,
    'mdp' => password_hash($mdp, PASSWORD_DEFAULT),
    'role' => $role,
]);
$idUser = $pdo->lastInsertId();
```

Cette requête ajoute le nouvel utilisateur. `password_hash()` transforme le mot de passe en un haché qu'on ne peut pas remettre en clair, et `lastInsertId()` récupère l'identifiant qui vient d'être créé, ce qui sert pour l'étape suivante.

Pour un compte refuge, on crée enfin son refuge en attente.

PHP — REFUGE EN ATTENTE

```
if ($role === 'refuge') {
    $pdo->prepare("INSERT INTO refuge (nom, ville, valide, id_utilisateur)
                VALUES (:nom, :ville, 0, :uid)")
    ->execute(['nom' => $nom_refuge, 'ville' => $ville_refuge, 'uid' =>
$idUser]);
}
```

Le refuge est ajouté avec `valide` à 0 et relié à l'utilisateur par son identifiant. Il reste en attente jusqu'à ce qu'un administrateur le valide.

3.3 Analyse des requêtes

L'inscription enregistre un utilisateur après avoir vérifié ses informations. Le mot de passe est protégé par un hachage, et la base empêche deux comptes d'avoir le même email. Pour les refuges, le fait de créer un refuge en attente évite que n'importe qui publie des annonces sans contrôle.

4 Connexion

4.1 Flux utilisateur

L'utilisateur entre son email et son mot de passe sur la page `login.php`. Quand il valide, les données partent au serveur. Si les identifiants sont bons, une session s'ouvre et il est renvoyé vers l'accueil. Sinon, un message d'erreur s'affiche.

4.2 Requête de vérification

PHP — VÉRIFICATION

```
$stmt = $pdo->prepare("SELECT * FROM utilisateur WHERE email = :email");
$stmt->execute(['email' => $email]);
$u = $stmt->fetch();

if ($u && password_verify($mdp, $u['mot_de_passe'])) {
    connecter($u);
    redirect('index.php');
} else {
    $erreurs[] = 'Email ou mot de passe incorrect.';
}
```

Une requête cherche l'utilisateur qui correspond à l'email entré. Si on en trouve un, on compare le mot de passe saisi avec le haché stocké grâce à `password_verify()`. Si ça correspond, la fonction `connecter()` ouvre la session et l'utilisateur est renvoyé vers son accueil.

PHP — CONNECTER()

```
function connecter(array $utilisateur): void {
    session_regenerate_id(true);
    $_SESSION['user'] = [
        'id' => (int) $utilisateur['id_utilisateur'],
        'prenom' => $utilisateur['prenom'],
        'role' => $utilisateur['role'],
    ];
}
```

Cette fonction change l'identifiant de session puis enregistre les infos utiles de l'utilisateur, dont son rôle, qui servira après pour le menu et pour les contrôles d'accès.

4.3 Analyse des requêtes

À la connexion, le script cherche l'utilisateur dans la base et vérifie le mot de passe sans jamais le déchiffrer. La requête est préparée, donc protégée contre les injections. Changer l'identifiant de session réduit le risque de vol de session. C'est simple mais ça suffit.

5 Recherche et consultation des animaux

5.1 Flux utilisateur

Depuis l'accueil ou le menu « Adopter », l'utilisateur arrive sur la page `recherche.php`. Il peut filtrer les animaux par espèce, par sexe, par ville, et par un texte libre sur le nom ou la race. Les résultats sortent sous forme de cartes, et un clic sur une carte ouvre la fiche détaillée de l'animal.

5.2 Requetes serveur

La requête de recherche est une des plus intéressantes du site. Elle croise trois tables et s'adapte aux filtres choisis.

PHP — RECHERCHE FILTRÉE

```
$sql = "SELECT a.id_animal, a.nom, a.race, a.age_annees, a.sexe, a.statut, a.photo,
           e.nom AS espece, e.emoji,
           COALESCE(r.ville, a.detenteur_ville) AS ville
FROM animal a
JOIN espece e      ON e.id_espece = a.id_espece
LEFT JOIN refuge r ON r.id_refuge = a.id_refuge
WHERE 1=1";

$params = [];

if ($f_espece) { $sql .= " AND a.id_espece = :espece"; $params['espece'] =
$f_espece; }
if ($f_sexe === 'M' || $f_sexe === 'F') { $sql .= " AND a.sexe = :sexe";
$params['sexe'] = $f_sexe; }
if ($f_ville !== '') { $sql .= " AND COALESCE(r.ville, a.detenteur_ville) LIKE
:ville"; $params['ville'] = "%$f_ville%"; }
if ($f_texte !== '') {
    $sql .= " AND (a.nom LIKE :q_nom OR a.race LIKE :q_race)";
    $params['q_nom'] = "%$f_texte%"; $params['q_race'] = "%$f_texte%";
}

$stmt = $pdo->prepare($sql);
$stmt->execute($params);
$animaux = $stmt->fetchAll();
```

On part d'une requête de base qui joint l'animal à son espèce et à son refuge, puis on ajoute une condition pour chaque filtre rempli. Toutes les valeurs entrées passent par des paramètres préparés, jamais collées directement dans le texte de la requête, ce qui évite les injections. Le `LEFT JOIN` sur le refuge, avec `COALESCE`, sert à afficher aussi les animaux détenus par des particuliers, qui n'ont pas de refuge. Sans ça, une jointure normale les ferait disparaître des résultats.

Sur la page d'accueil, une autre requête compte les animaux disponibles par espèce, pour l'afficher sur chaque carte.

SQL

```
SELECT e.id_espece, e.nom, e.emoji, e.image,  
       COUNT(a.id_animal) AS nb  
FROM espece e  
LEFT JOIN animal a ON a.id_espece = e.id_espece AND a.statut = 'disponible'  
GROUP BY e.id_espece;
```

Elle utilise une jointure externe et un regroupement pour ramener, pour chaque espèce, le nombre d'animaux encore disponibles.

5.3 Analyse des requêtes

La recherche montre comment construire une requête qui s'adapte tout en restant sûre, puisque les valeurs restent dans des paramètres préparés. Le choix du `LEFT JOIN` avec `COALESCE` est utile : il gère dans une seule requête les animaux des refuges et ceux des particuliers, sans en oublier.

6 Publier une annonce

6.1 Flux utilisateur

Un refuge validé arrive sur la page `annonce-form.php` depuis « Mes annonces ». Il entre les infos de l'animal (nom, espèce, race, âge, sexe, stérilisation, ancien propriétaire, statut, description) et il peut ajouter une photo. La même page sert aussi à modifier une annonce. Si le refuge n'est pas encore validé, un message lui dit qu'il doit attendre l'accord de l'administrateur avant de publier.

Un particulier utilise la même page, mais sans choisir de refuge : il indique juste sa ville, et l'animal est rattaché à son compte (la colonne `id_proprietaire`). Il n'a pas besoin de validation, il peut publier directement. Le reste du formulaire est identique.

6.2 Requêtes serveur

D'abord, on récupère seulement les refuges qui appartiennent à l'utilisateur et qui sont validés, pour l'empêcher de publier pour un autre refuge.

PHP — REFUGES DE L'UTILISATEUR

```
$stmt = $pdo->prepare("SELECT id_refuge, nom FROM refuge
                        WHERE id_utilisateur = :uid AND valide = 1 ORDER BY nom");
$stmt->execute(['uid' => user()['id']]);
$refuges = $stmt->fetchAll();
```

Ensuite, si une photo a été envoyée, on la vérifie avant de l'enregistrer.

PHP — UPLOAD PHOTO

```
if (!empty($_FILES['photo']['name']) && $_FILES['photo']['error'] === UPLOAD_ERR_OK)
{
    $info = @getimagesize($_FILES['photo']['tmp_name']);
    $types =
['image/jpeg'=>'jpg', 'image/png'=>'png', 'image/webp'=>'webp', 'image/gif'=>'gif'];
    if (!$info || !isset($types[$info['mime']])) {
        $erreurs[] = 'La photo doit etre une image (jpg, png, webp ou gif).';
    } elseif ($_FILES['photo']['size'] > 3 * 1024 * 1024) {
        $erreurs[] = 'La photo est trop lourde (max 3 Mo).';
    } else {
        $fname = 'animal_' . uniqid() . '.' . $types[$info['mime']];
        move_uploaded_file($_FILES['photo']['tmp_name'], __DIR__ . '/img/uploads/' .
        $fname);
        $animal['photo'] = 'img/uploads/' . $fname;
    }
}
```

On vérifie le vrai type de l'image avec `getimagesize()`, et pas seulement son extension, puis sa taille. Si c'est bon, le fichier est renommé avec un nom unique et déplacé dans le dossier `img/uploads/`, et on garde son chemin pour l'enregistrer en base.

Enfin, l'annonce est ajoutée.

PHP — INSERTION ANIMAL

```
$sql = "INSERT INTO animal (nom, id_espece, id_refuge, race, age_annees, sexe,
                           sterilise, ancien_proprietaire, description, statut,
                           photo)
      VALUES (:nom, :id_espece, :id_refuge, :race, :age, :sexe,
              :sterilise, :ancien, :desc, :statut, :photo)";
$pdo->prepare($sql)->execute($params);
```

Cette requête enregistre le nouvel animal avec toutes ses infos et le chemin de sa photo.

6.3 Analyse des requêtes

La publication montre l'écriture et la modification de données, avec une vérification de propriété : un refuge ne peut publier que pour un refuge qui est à lui et qui est validé. L'envoi de la photo est sécurisé parce qu'on contrôle son vrai type et sa taille, ce qui empêche d'envoyer un fichier qui ne serait pas une image.

7 Demande d'adoption

7.1 Flux utilisateur

Sur la fiche d'un animal disponible, un utilisateur connecté clique sur « Faire une demande d'adoption », écrit un message au refuge et l'envoie. Il retrouve ensuite sa demande dans la page « Mes demandes ».

7.2 Requête serveur

PHP — CRÉATION DE LA DEMANDE

```
if (strlen($message) < 10) {
    $erreurs[] = 'Merci d\'ecrire quelques mots (10 caracteres minimum) pour le
    refuge.';
} else {
    $check = $pdo->prepare("SELECT 1 FROM demande_adoption
                           WHERE id_animal = :a AND id_utilisateur = :u");
    $check->execute(['a' => $id, 'u' => user()['id']]);
    if ($check->fetch()) {
        $erreurs[] = 'Vous avez déjà envoye une demande pour cet animal.';
    } else {
        $pdo->prepare("INSERT INTO demande_adoption (id_animal, id_utilisateur,
        message)
                     VALUES (:a, :u, :m)")
        ->execute(['a' => $id, 'u' => user()['id'], 'm' => $message]);
    }
}
```

On vérifie d'abord que le message fait au moins dix caractères. Ensuite, une requête regarde si l'utilisateur n'a pas déjà fait une demande sur cet animal, pour éviter les doublons. Si c'est bon, la demande est ajoutée et reliée tout seul à l'utilisateur connecté.

7.3 Analyse des requêtes

Cette partie relie un utilisateur et un animal dans la table `demande_adoption`. On vérifie la longueur du message et on empêche les doublons avec une requête de contrôle avant l'ajout. L'identifiant de l'utilisateur vient de la session, donc la demande est bien rattachée à la bonne personne.

8 Gestion des demandes

8.1 Flux utilisateur

La page « Mes demandes » a deux parties. Pour un refuge ou un particulier, elle affiche d'abord les demandes reçues sur ses animaux, avec un bouton pour accepter et un pour refuser. Ensuite, pour tout le monde, elle affiche les demandes qu'on a envoyées soi-même, avec leur statut. Quand une demande est acceptée, l'animal passe tout seul au statut « réservé ».

8.2 Requêtes serveur

La requête qui liste les demandes reçues croise quatre tables.

SQL — DEMANDES REÇUES

```
SELECT d.id_demande, d.message, d.statut, d.date_demande,
       a.id_animal, a.nom AS animal, r.nom AS refuge,
       u.prenom, u.nom, u.email
FROM demande_adoption d
JOIN animal a      ON a.id_animal = d.id_animal
JOIN refuge r      ON r.id_refuge = a.id_refuge
JOIN utilisateur u ON u.id_utilisateur = d.id_utilisateur
WHERE r.id_utilisateur = :uid
ORDER BY d.statut = 'en_attente' DESC, d.date_demande DESC;
```

Elle récupère, pour le refuge connecté, toutes les demandes faites sur ses animaux, avec le nom du demandeur et son message. Au moment d'accepter ou de refuser, on vérifie d'abord que la demande concerne bien un animal de ce refuge.

PHP — ACCEPTER / REFUSER

```
$chk = $pdo->prepare("SELECT a.id_animal, r.id_utilisateur
                    FROM demande_adoption d
                    JOIN animal a ON a.id_animal = d.id_animal
                    LEFT JOIN refuge r ON r.id_refuge = a.id_refuge
                    WHERE d.id_demande = :id");
$chk->execute(['id' => $idd]);
$row = $chk->fetch();

if ($row && (int) $row['id_utilisateur'] === $uid) {
    $pdo->prepare("UPDATE demande_adoption SET statut = :s WHERE id_demande = :id")
    ->execute(['s' => $action, 'id' => $idd]);
    if ($action === 'acceptee') {
        $pdo->prepare("UPDATE animal SET statut = 'reserve' WHERE id_animal = :a")
        ->execute(['a' => $row['id_animal']]);
    }
}
```

Ce code vérifie que la demande porte bien sur un animal du refuge de l'utilisateur connecté. Si c'est le cas, le statut de la demande est changé, et quand elle est acceptée, l'animal passe au statut « réservé ».

8.3 Analyse des requêtes

C'est ici qu'on a les pages qui touchent à plusieurs tables en même temps, jusqu'à quatre. La sécurité compte beaucoup : avant chaque action, on s'assure que la demande porte sur un animal du refuge de l'utilisateur, comme ça un refuge ne peut pas traiter les demandes d'un autre. Accepter une demande change aussi le statut de l'animal pour que tout reste cohérent.

9 Administration

9.1 Flux utilisateur

La page `admin.php` est réservée à l'administrateur. Elle affiche des statistiques sur le site, la liste des refuges en attente de validation avec un bouton pour valider et un pour rejeter, et toutes les demandes du site.

9.2 Requêtes serveur

Les statistiques viennent de plusieurs comptages.

PHP — STATISTIQUES

```
$stats = [  
    'animaux'      => $pdo->query("SELECT COUNT(*) FROM animal")->fetchColumn(),  
    'disponibles' => $pdo->query("SELECT COUNT(*) FROM animal WHERE  
statut='disponible'")->fetchColumn(),  
    'refuges'      => $pdo->query("SELECT COUNT(*) FROM refuge")->fetchColumn(),  
    'membres'      => $pdo->query("SELECT COUNT(*) FROM utilisateur")->fetchColumn(),  
    'demandes'     => $pdo->query("SELECT COUNT(*) FROM demande_adoption WHERE  
statut='en_attente'")->fetchColumn(),  
];
```

Valider un refuge se fait par une simple mise à jour, et le rejeter par une suppression.

PHP — VALIDER / REJETER

```
if ($_POST['refuge_action'] === 'valider') {  
    $pdo->prepare("UPDATE refuge SET valide = 1 WHERE id_refuge = :id")->  
>execute(['id' => $idr]);  
} elseif ($_POST['refuge_action'] === 'rejeter') {  
    $pdo->prepare("DELETE FROM refuge WHERE id_refuge = :id")->execute(['id' =>  
$idr]);  
}
```

Valider un refuge, ça revient à passer sa colonne `valide` à 1, et là il peut publier des annonces. Le rejet supprime juste le refuge en attente.

9.3 Analyse des requêtes

L'accès à cette page est contrôlé dès le début : un utilisateur qui n'est pas administrateur est renvoyé ailleurs. L'administrateur voit toutes les demandes, alors que la page « Mes demandes » reste personnelle. Faire bien la différence entre ces deux cas nous a demandé un peu d'attention.

10 Procédure de lancement

Pour lancer le site en local avec XAMPP, on copie le contenu du dossier `htdocs/` du projet dans `xampp/htdocs/adoption/`, puis on démarre Apache et MySQL depuis le panneau XAMPP. On ouvre ensuite `http://localhost/phpmyadmin`, on importe le fichier `dump-adoption-mahroug-benfakir.sql` depuis l'onglet « Importer », ce qui crée la base `adoption` avec ses données de test. Il reste juste à ouvrir `http://localhost/adoption/`. Le fichier `connexion.php` reconnaît l'environnement local et prend tout seul l'utilisateur `root` avec un mot de passe vide (le réglage de XAMPP par défaut), donc il n'y a rien à changer.

Le site est aussi déjà en ligne à l'adresse ines-rihane-adoption-animal.site.je.

11 Choix techniques et sécurité

On a utilisé PDO avec des requêtes préparées pour toutes les requêtes qui reçoivent des données de l'utilisateur, ce qui enlève le risque d'injection SQL. Les mots de passe sont hachés avec `password_hash()` et vérifiés avec `password_verify()`. Ce qui est affiché à l'écran est échappé avec `htmlspecialchars()` à travers une fonction `e()`, pour se protéger des attaques XSS. La connexion se fait avec les sessions, et on change l'identifiant de session au moment de se connecter. Les rôles et la propriété des données sont contrôlés avec les fonctions `exiger_connexion()` et `exiger_role()`, et par des vérifications avant chaque modification, comme ça un refuge ne touche qu'à ses propres données. Les données des formulaires sont vérifiées côté serveur. La carte des refuges a été faite avec Leaflet et OpenStreetMap, sans clé d'API, à partir des coordonnées enregistrées en base.

12 Organisation et répartition du travail

On a fait le projet en binôme, en se partageant les tâches mais en décidant ensemble des points importants comme le modèle de données et les choix techniques. Rihane s'est occupée surtout de la conception de la base et des données de test, de la recherche et des fiches animaux, de la gestion des demandes et du design. Inès a travaillé surtout sur la connexion et les rôles, la

publication des annonces avec l'upload de photo, la validation des refuges et l'administration, et aussi sur la mise en ligne et les tests. Le rapport a été écrit à deux. Chacune est quand même passée sur l'ensemble du projet, avec une personne en charge principale par bloc.

13 Apports de l'intelligence artificielle

On a utilisé une intelligence artificielle comme outil d'aide pendant le développement, en gardant la main sur les choix et en relisant ce qui était produit. Elle nous a aidées à écrire plus vite certaines parties répétitives du code, comme les formulaires ou les requêtes, qu'on a ensuite adaptées et comprises. Elle nous a aussi aidées à corriger des erreurs, par exemple des problèmes d'encodage, de version de PHP ou de configuration de la base en ligne. Elle nous a servi à organiser le rapport. On a fait attention à ne pas copier du code sans le comprendre et à ne pas la laisser décider de la conception à notre place. Chaque fonctionnalité a été testée par nous, et l'IA a surtout servi d'aide.

14 Conclusion

Ce projet nous a fait réaliser, du début à la fin, une application web reliée à une base de données : la conception du modèle, la création de la base, le développement des pages, la sécurité, puis la mise en ligne. Toutes les fonctionnalités demandées sont là, avec en plus quelques options comme la vérification des formulaires, les pages qui croisent plusieurs tables, la connexion par compte et la mise en ligne.

On pourrait encore l'améliorer, par exemple avec une messagerie entre l'adoptant et le refuge une fois la demande acceptée, un système de favoris, l'envoi d'emails quand une demande arrive ou est acceptée, ou la possibilité de mettre plusieurs photos par animal.